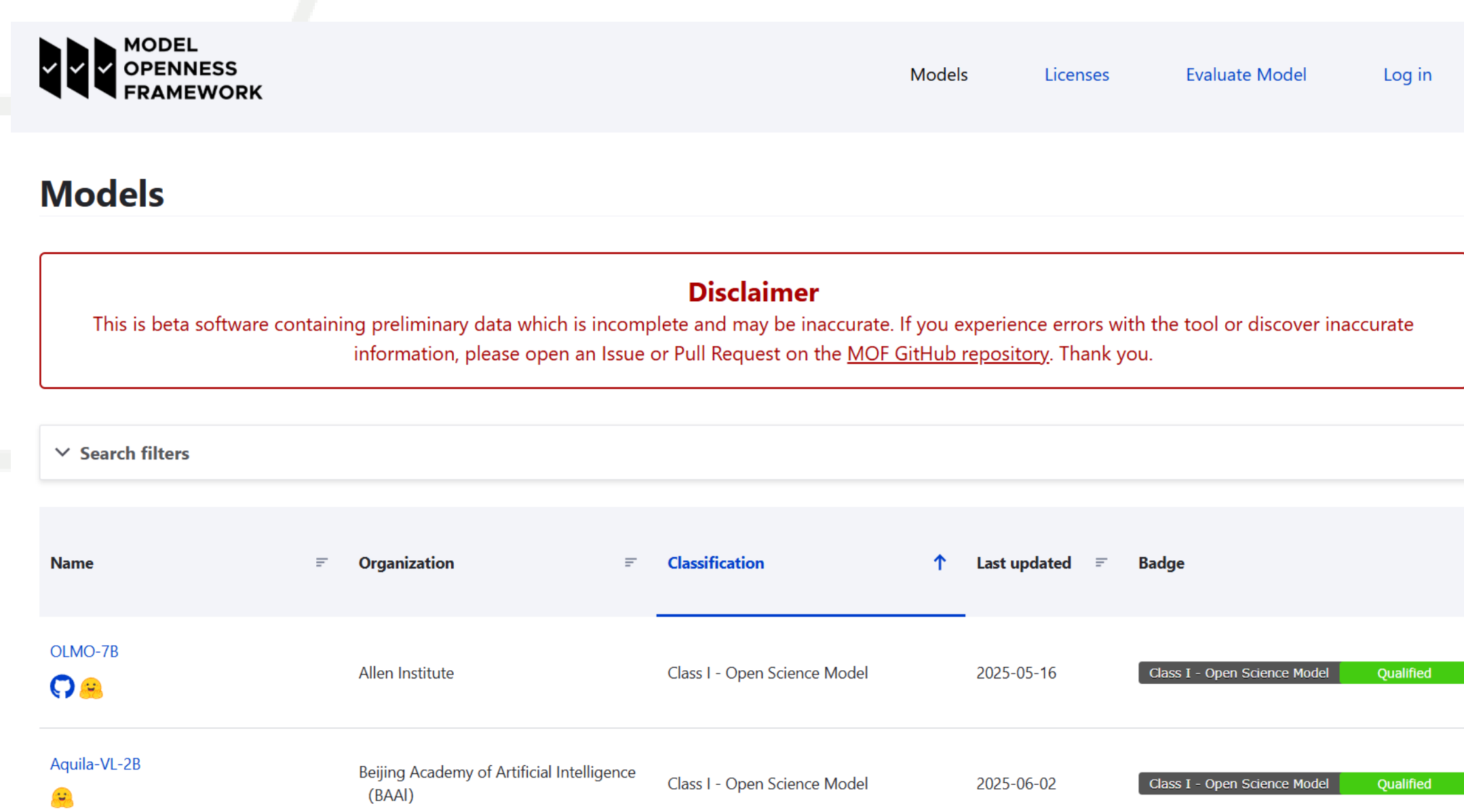


Project Overview

The Model Openness Framework is a framework designed to rank machine learning/AI models based on: Completeness – the number of standard model components included and Openness – the licensing of components with open-source licenses.



This framework is implemented via the Model Openness Tool (MOT) at mot.isitopen.ai.

The benefits of this framework include combating “openwashing”, promoting reproducibility, enhancing transparency, and standardization.

Goals and Milestones: Vishank

Develop a basic test suite for the MOT:

1. Produce a comprehensive set of test model files
2. Develop a script to generate further test model files as needed
3. Create a script to evaluate the test model files

Goals and Milestones: Jake

Develop a basic AI pipeline for Model data :

1. Make a simple RAG AI pipeline running on Ollama Granite.
2. Develop a prompt to generate model yaml file from the Pipeline and fetched data.
3. Create a Python script to fetch the model data online.

Highlights and Accomplishments: Test Suite

1. Test Model Files

model openness tool / Test_Data /
Class1TestFile_C1_100%-C2_100%-C3_100%-15C_2G_15L_15V_0L_15T.yaml
OnlyGlobalLicense_C1_33%-C2_27%-C3_17%-5C_4G_0L_5V_0L_5T.yaml
NoLicenseFile_C1_0%-C2_0%-C3_0%-9C_0G_0L_0V_0L_0T.yaml
OnlyComponentLicenses_C1_13%-C2_18%-C3_33%-8C_0G_8L_2V_6L_2T.yaml
RandomTestFile_C1_7%-C2_9%-C3_0%-6C_2G_6L_1V_5L_1T.yaml

- Test model files are named to match their expected evaluation
- 27 test model files available covering various aspects regarding class evaluations, components, licenses, etc.

2. Test Model File Generation

```
usage: generate-test-files.py [-h] [-c NUM_COMPONENTS] [-g NUM_GLOBAL_LICENSES] [-i NUM_INVALID_LICENSES] [-v NUM_VALID_LICENSES] [-t NUM_TYPE_APPROPRIATE_LICENSES] [-d DATE] [-by TYPE] [-a ARCHITECTURE] [-p PRODUCER] [-co CONTACT] [-r REPOSITORY] [-hf HUGGINGFACE] [-sp SAVE_PATH]

Generate test files for the Model Openness Tool.

options:
  -h, --help            show this help message and exit
  -c, --num-components NUM_COMPONENTS
                        Number of components in the test file.
  -g, --num-global-licenses NUM_GLOBAL_LICENSES
                        Number of global licenses in the test file.
  -i, --num-invalid-licenses NUM_INVALID_LICENSES
                        Number of invalid licenses in the test file.
  -v, --num-valid-licenses NUM_VALID_LICENSES
                        Number of valid licenses in the test file.
  -t, --num-type-appropriate-licenses NUM_TYPE_APPROPRIATE_LICENSES
                        Number of type-appropriate licenses in the test file.
  -d, --date DATE        Date to use for the test file.
  -by, --by TYPE        Type to use for the test file.
  -a, --architecture ARCHITECTURE
                        Architecture to use for the test file.
  -p, --producer PRODUCER
                        Producer to use for the test file.
  -co, --contact CONTACT
                        Contact to use for the test file.
  -r, --repository REPOSITORY
                        Repository to use for the test file.
  -hf, --huggingface HUGGINGFACE
                        HuggingFace repository to use for the test file.
  -sp, --save-path SAVE_PATH
                        Path to save the test files.
```

Sample Command:

```
$../generate-test-files.py -c 11 -g 2 -l 11 -v 5 -i 6 -t 5 -n "RandomTestFile"
```

Sample Output:

```
framework:
  name: Model Openness Framework
  version: '1.0'
  date: '2024-12-15'
  release:
    name: RandomTestFile_C1_27%-C2_18%-C3_17%-11C_2G_11L_11V_0L_11T
    version: 4B
    date: '2025-06-17'
    type: multimodal
    architecture: RNN
    origin: Pre-Test
    producer: Test2
```

- Generation script produces tailored files for specific test cases
- Output files are automatically named with their expected evaluation

3. Test Model File Evaluation

```
$./vendor/bin/drush scr scripts/test-model-files.php
Class1TestFile_C1_100%-C2_100%-C3_100%-15C_2G_15L_15V_0L_15T.yaml: PASS
Class2TestFile_C1_75%-C2_100%-C3_100%-11C_1G_11L_11V_0L_11T.yaml: FAIL
Expected: {"class_1_progress":75,"class_2_progress":100,"class_3_progress":100,"num_components":11,"num_global_licenses":1,"num_component_licenses":11,"num_valid_licenses":11,"num_invalid_licenses":0,"num_type_appropriate_licenses":11}
Actual: {"class_1_progress":77,"class_2_progress":100,"class_3_progress":100,"num_components":11,"num_global_licenses":1,"num_component_licenses":11,"num_valid_licenses":11,"num_invalid_licenses":0,"num_type_appropriate_licenses":11}
Class2TestFile_C1_71%-C2_100%-C3_100%-11C_1G_11L_11V_0L_11T.yaml: PASS
```

- Checks if the model evaluation by the MOT matches the expected evaluation given by the name
- Produces a diagnostic message if the expected evaluation differs from the actual evaluation

Highlights and Accomplishments AI Pipeline

1. Python Script for Fetch Data

```
model_name = input("model: ").strip()
url = f"https://huggingface.co/api/models/{model_name}"

response = requests.get(url)
print("Status code:", response.status_code)

if response.status_code == 200:
    data = response.json()

    # Convert to pretty JSON string
    metadata_text = json.dumps(data, indent=2)

    # Save to a text file
    with open("model_metadata.txt", "w", encoding="utf-8") as f:
        f.write(metadata_text)

    print("Metadata saved to model_metadata.txt")
else:
    print("Error fetching model info")

✓ 3.4s

Status code: 200
Metadata saved to model_metadata.txt
```

- Python script fetch the metadata directly from the respective Hugging face repo.
- Effectively fetch model licensing and parameters in structured format.

2. Model Prompt & RAG Pipeline

Sample Input: "mistralai/Mistral-7B-Instruct-v0.2"

Query: "What is Research Paper used?"

```
name: 'Research paper'
description: 'Research paper detailing the development and capabilities of the model'
license: ''
license_path: 'arxiv:2310.06825'
component_path: 'https://arxiv.org/abs/2310.06825'
```

Sample Output YAML:

```
license:
  distribution:
    name: 'Apache 2.0'
    path: 'license:apache-2.0'
  code:
    name: 'Apache 2.0'
    path: 'license:apache-2.0'
  type: 'text-generation'
  architecture: 'MistralForCausalLM'
  origin: 'mistralai'
  producer: 'Mistral AI'
  contact: ''
components:
  - name: 'Model architecture'
    description: 'Well commented code for the model's architecture'
    license: 'unlicensed'
    component_path: 'transformersInfo.auto_model: AutoModelForCausalLM'

  - name: 'Supporting libraries and tools'
    description: 'Libraries and tools used in the model's development (transformers, PyTorch)'
    license: 'Apache 2.0'
    license_path: 'license:apache-2.0'
    component_path: 'transformers, pytorch'

  - name: 'Model parameters (Final)'
    description: 'Trained model parameters, weights and biases; stored in .safetensors files'
    license: ''
    license_path: ''
    component_path: 'model-00001-of-00003.safetensors, model-00002-of-00003.safetensors, model-00003-of-00003.safetensors'

  - name: 'Model metadata'
    description: 'Training configuration and optimizer states; part of the Hugging Face model card'
    license: 'Apache 2.0'
    license_path: 'license:apache-2.0'
    component_path: 'cardData'
```

Open Source Outcomes

- Increased efficiency of contributions via testing
- Testing reduces barrier of entry for new contributors
- Testing enhances the reliability and trustworthiness of the MOT
- Increased efficiency of generating model YAML data and add new model to the MOT
- Easy portability and can be run locally on resource constraint device.
- Consistent output compared to the commercial LLM model.

Future Work

1. Continue developing the test suite: increase coverage of test model files, add checks beyond model evaluation, add more customizability to test generation script, etc.
2. Improve meta data collection to support more source to draw data from.
3. Continue developing the pipeline so that it can produce more complex YAML file like those produced from the commercial LLM with the improved prompt.